

Constructing Canonical LR(1) Parsing Tables

- ◆ In SLR method, the state i makes a reduction by $A \rightarrow \alpha$ when the current token is a :
 - if the $A \rightarrow \alpha.$ in the I_i and a is $\text{FOLLOW}(A)$
- ◆ In some situations, βA cannot be followed by the terminal a in a right-sentential form when $\beta\alpha$ and the state i are on the top stack. This means that making reduction in this case is not correct.

$S \rightarrow AaAb$

$S \Rightarrow AaAb \Rightarrow Aab \Rightarrow ab$

$S \Rightarrow BbBa \Rightarrow Bba \Rightarrow ba$

$S \rightarrow BbBa$

$A \rightarrow \epsilon$

$AaAb \Rightarrow Aa \epsilon b$

$BbBa \Rightarrow Bb \epsilon a$

$B \rightarrow \epsilon$

$Aab \Rightarrow \epsilon ab$

$Bba \Rightarrow \epsilon ba$

LR(1) Item

- ◆ To avoid some of invalid reductions, the states need to carry more information.
- ◆ Extra information is put into a state by including a terminal symbol as a second component in an item.
- ◆ A LR(1) item is:

$A \rightarrow \alpha.\beta, a$ where a is the look-head
of the LR(1) item
(a is a terminal or end-marker.)

LR(1) Item (cont.)

- ◆ When β (in the LR(1) item $A \rightarrow \alpha.\beta, a$) is not empty, the look-head does not have any affect.
- ◆ When β is empty ($A \rightarrow \alpha., a$), we do the reduction by $A \rightarrow \alpha$ only if the next input symbol is a (not for any terminal in $\text{FOLLOW}(A)$).
- ◆ A state will contain $A \rightarrow \alpha., a_1$ where $\{a_1, \dots, a_n\} \subseteq \text{FOLLOW}(A)$

...

$A \rightarrow \alpha., a_n$

Canonical Collection of Sets of LR(1) Items



The construction of the canonical collection of the sets of LR(1) items are similar to the construction of the canonical collection of the sets of LR(0) items, except that *closure* and *goto* operations work a little bit different.

$\text{closure}(I)$ is: (where I is a set of LR(1) items)

- every LR(1) item in I is in $\text{closure}(I)$
- if $A \rightarrow \alpha \cdot B \beta$, a in $\text{closure}(I)$ and $B \rightarrow \gamma$ is a production rule of G ; then $B \rightarrow \cdot \gamma$, b will be in the $\text{closure}(I)$ for each terminal b in $\text{FIRST}(\beta a)$.

goto operation

If I is a set of LR(1) items and X is a grammar symbol (terminal or non-terminal), then $\text{goto}(I, X)$ is defined as follows:

- If $A \rightarrow \alpha.X\beta$, a in I
then every item in $\text{closure}(\{A \rightarrow \alpha X.\beta, a\})$ will be in $\text{goto}(I, X)$.

Construction of The Canonical LR(1) Collection

- *Algorithm:*

C is { **closure**($\{S' \rightarrow .S, \$\}$) }

repeat the followings until no more set of LR(1) items can be added to C .

for each I in C and each grammar symbol X

if $\text{goto}(I, X)$ is not empty and not in C

add $\text{goto}(I, X)$ to C

- goto function is a DFA on the sets in C .

A Short Notation for The Sets of LR(1) Items



A set of LR(1) items containing the following items

$$A \rightarrow \alpha.\beta, a_1$$

...

$$A \rightarrow \alpha.\beta, a_n$$

can be written as

$$A \rightarrow \alpha.\beta, a_1/a_2/.../a_n$$

Canonical LR(1) Collection - Example1

1) $S \rightarrow AaAb$

2) $S \rightarrow BbBa$

3) $A \rightarrow \epsilon$

4) $B \rightarrow \epsilon$

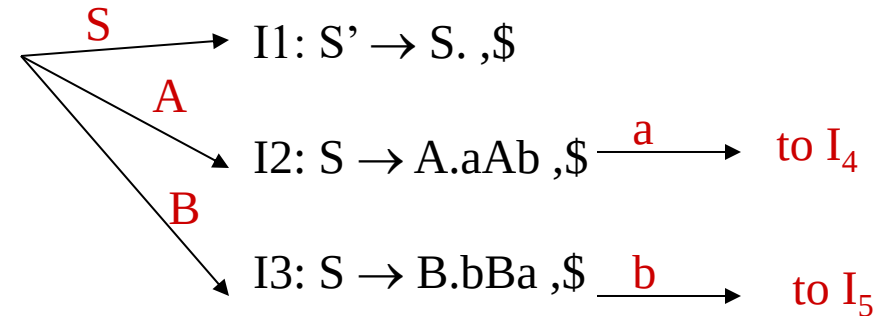
$I_0: S' \rightarrow .S, \$$

$S \rightarrow .AaAb, \$$

$S \rightarrow .BbBa, \$$

$A \rightarrow ., a$

$B \rightarrow ., b$



$I_4: S \rightarrow Aa.Ab, \$ \xrightarrow{A} I_6: S \rightarrow AaA.b, \$ \xrightarrow{b} I_8: S \rightarrow AaAb., \$$
 $A \rightarrow ., b$

$I_5: S \rightarrow Bb.Ba, \$ \xrightarrow{B} I_7: S \rightarrow BbB.a, \$ \xrightarrow{a} I_9: S \rightarrow BbBa., \$$
 $B \rightarrow ., a$

Construction of LR(1) Parsing Tables

1. Construct the canonical collection of sets of LR(1) items for G' . $C \leftarrow \{I_0, \dots, I_n\}$
2. Create the parsing action table as follows
 - If a is a terminal, $A \rightarrow \alpha.a\beta$, b in I_i and $\text{goto}(I_i, a) = I_j$ then action $[i, a]$ is *shift j*.
 - If $A \rightarrow \alpha.$, a is in I_i , then action $[i, a]$ is *reduce $A \rightarrow \alpha$* where $A \neq S'$.
 - If $S' \rightarrow S.$, $\$$ is in I_i , then action $[i, \$]$ is *accept*.
 - If any conflicting actions generated by these rules, the grammar is not LR(1).
3. Create the parsing goto table
 - for all non-terminals A , if $\text{goto}(I_i, A) = I_j$ then goto $[i, A] = j$
4. All entries not defined by (2) and (3) are errors.
5. Initial state of the parser contains $S' \rightarrow .S, \$$

Cf. p227 SLR
parsing table

LR(1) Parsing Tables – (for Example1)

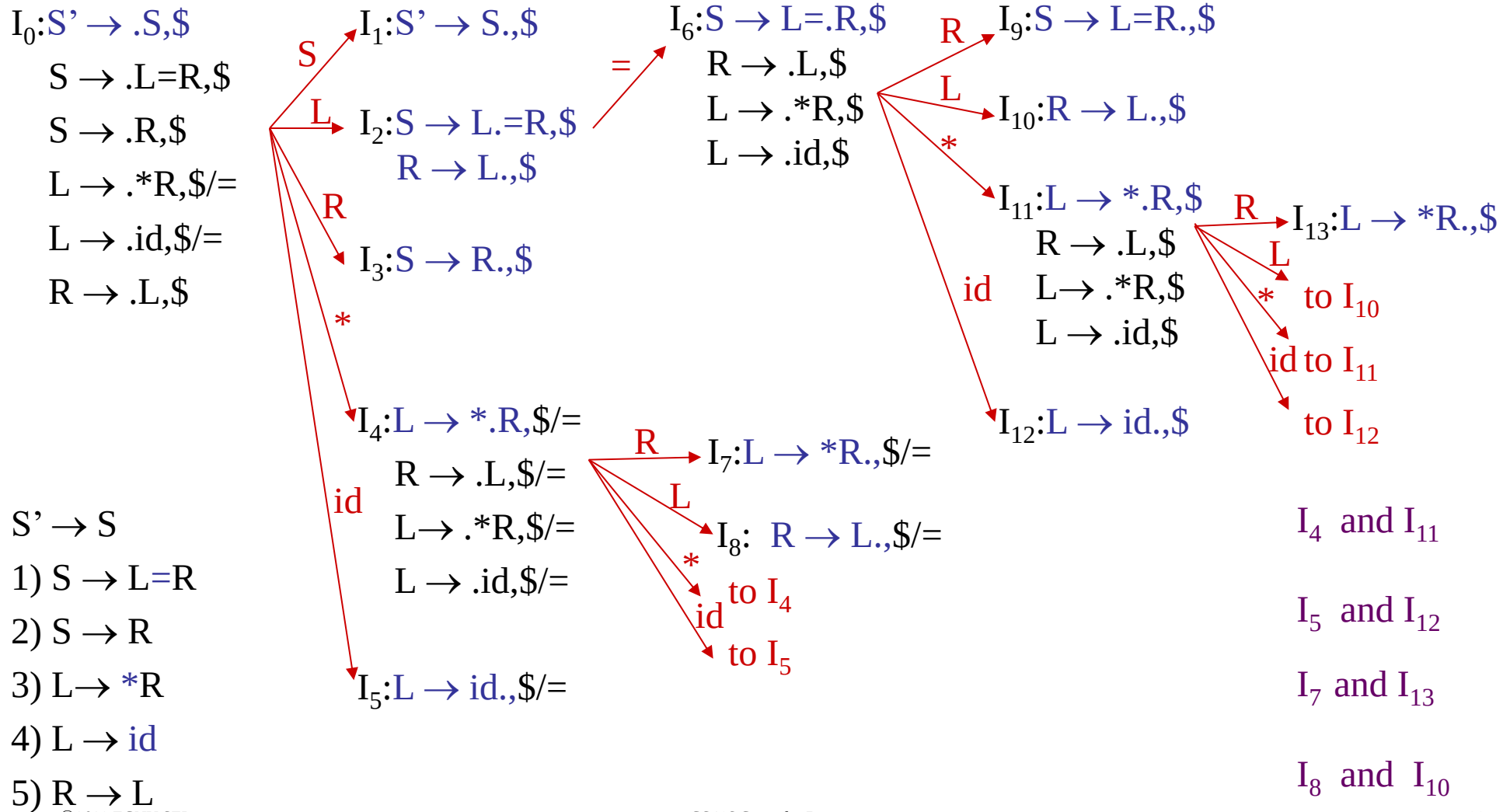
	a	b	\$		S	L	R
0	r3	r4			1	2	3
1			acc				
2	s4						
3		s5					
4		r3				6	
5	r4						7
6		s8					
7	s9						
8			r1				
9			r2				

no shift/reduce or
no reduce/reduce conflict



so, it is a LR(1) grammar

Canonical LR(1) Collection - Example2



LR(1) Parsing Tables – (for Example2)

	id	*	=	\$	S	L	R
0	s5	s4			1	2	3
1				acc			
2			s6	r5			
3				r2			
4	s5	s4				8	7
5			r4	r4			
6	s12	s11				10	9
7			r3	r3			
8			r5	r5			
9				r1			
10				r5			
11	s12	s11				10	13
12				r4			
13				r3			

no shift/reduce or
no reduce/reduce conflict



so, it is a LR(1) grammar

LALR Parsing Tables (P.236)

- ◆ LALR stands for LookAhead LR.
- ◆ LALR parsers are often used in practice because LALR parsing tables are smaller than LR(1) parsing tables.
- ◆ The number of states in SLR and LALR parsing tables for a grammar G are equal.
- ◆ But LALR parsers recognize more grammars than SLR parsers.
- ◆ YACC creates a LALR parser for the given grammar.
- ◆ A state of LALR parser will be again a set of LR(1) items.

The Core of A Set of LR(1) Items

- ◆ The core of a set of LR(1) items is the set of its first component.

Ex: $S \rightarrow L \bullet = R, \$$ $\rightarrow$ $S \rightarrow L \bullet = R$
 $R \rightarrow L \bullet, \$$ $R \rightarrow L \bullet$ $\leftarrow$ **core**

- ◆ We will find the states (sets of LR(1) items) in a canonical LR(1) parser with **same cores** (同核). Then we will merge them as a single state.

$I_1: L \rightarrow id \bullet, =$ A new state: $I_{12}: L \rightarrow id \bullet, =$
 $\rightarrow$ $L \rightarrow id \bullet, \$$
 $I_2: L \rightarrow id \bullet, \$$ have same core, merge them or $I_{12}: L \rightarrow id \bullet, =/\$$

- ◆ We will do this for all states of a canonical LR(1) parser to get the states of the LALR parser.
- ◆ In fact, the number of the states of the LALR parser for a grammar will be equal to the number of states of the SLR parser for that grammar.

Creation of LALR Parsing Tables

- ◆ Create the canonical LR(1) collection of the sets of LR(1) items for the given grammar.
- ◆ Find each core; find all sets having that same core; replace those sets having same cores with a single set which is their union.

$$C=\{I_0,...,I_n\} \rightarrow C'=\{J_1,...,J_m\} \quad \text{where } m \leq n$$

- ◆ Create the parsing tables (action and goto tables) same as the construction of the parsing tables of LR(1) parser.

Note that: If $J=I_1 \cup \dots \cup I_k$ since $I_1,...,I_k$ have same cores

$\rightarrow$ cores of $\text{goto}(I_1,X), \dots, \text{goto}(I_k,X)$ must be same.

So, $\text{goto}(J,X)=K$ where K is the union of all sets of items having same cores as $\text{goto}(I_1,X)$.

- ◆ If no conflict is introduced, the grammar is LALR(1) grammar. (We may only introduce reduce/reduce conflicts; we cannot introduce a shift/reduce conflict)



Creating LALR Parsing Tables



- ◆ This shrink process may introduce a reduce/reduce? conflict in the resulting LALR parser (so the grammar is NOT LALR)
- ◆ But, this shrink process does not produce a shift/reduce conflict.

Shift/Reduce Conflict

- ◆ We say that we cannot introduce a shift/reduce conflict during the shrink process for the creation of the states of a LALR parser.
- ◆ Assume that we can introduce a shift/reduce conflict. In this case, a state of LALR parser must have:

$$A \rightarrow \alpha \bullet, a \quad \text{and} \quad B \rightarrow \beta \bullet a \gamma, b$$

- ◆ This means that a state of the canonical LR(1) parser must have:

$$A \rightarrow \alpha \bullet, a \quad \text{and} \quad B \rightarrow \beta \bullet a \gamma, c$$

But, this state has also a shift/reduce conflict. i.e. The original canonical LR(1) parser has a conflict.

(Reason for this, the shift operation does not depend on lookaheads)

Reduce/Reduce Conflict

But, we may introduce a reduce/reduce conflict during the shrink process for the creation of the states of a LALR parser.

$$I_1 : A \rightarrow \alpha \bullet, a$$

$$B \rightarrow \beta \bullet, b$$

$$I_2 : A \rightarrow \alpha \bullet, b$$

$$B \rightarrow \beta \bullet, c$$



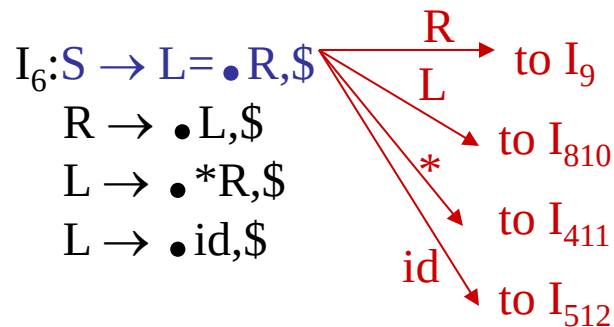
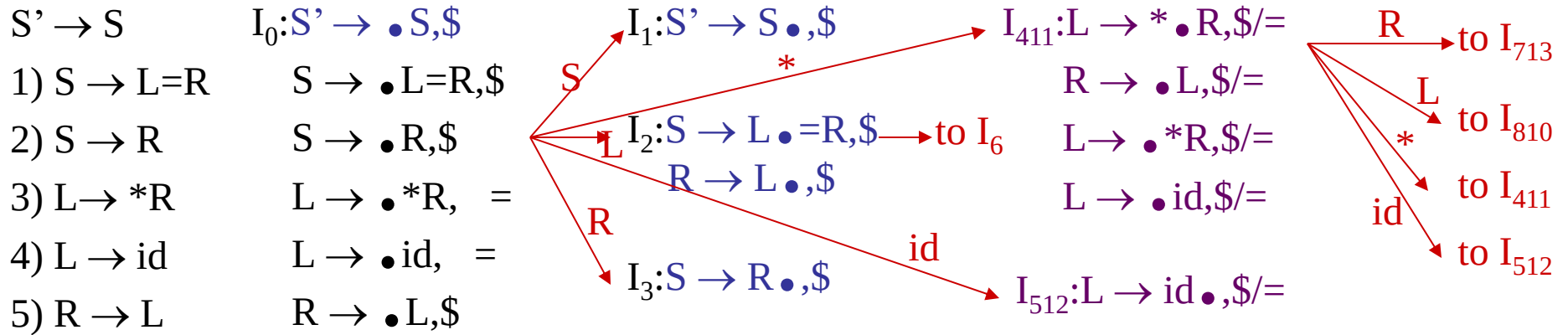
$$I_{12} : A \rightarrow \alpha \bullet, a/b$$

➔ reduce/reduce

$$B \rightarrow \beta \bullet, b/c$$

conflict

Canonical LALR(1) Collection – Example 2



$I_9: S \rightarrow L=R \bullet, \$$

Same Cores
 I_4 and I_{11}

I_5 and I_{12}

I_7 and I_{13}

I_8 and I_{10}

$I_{713}: L \rightarrow *R \bullet, \$/=$

$I_{810}: R \rightarrow L \bullet, \$/=$

LALR(1) Parsing Tables - (for Example2)

	id	*	=	\$		S	L	R
0	s512	s411				1	2	3
1				acc				
2			s6	r5				
3				r2				
4	s512	s411					810	713
5			r4	r4				
6	s512	s411					810	9
7			r3	r3				
8			r5	r5				
9				r1				

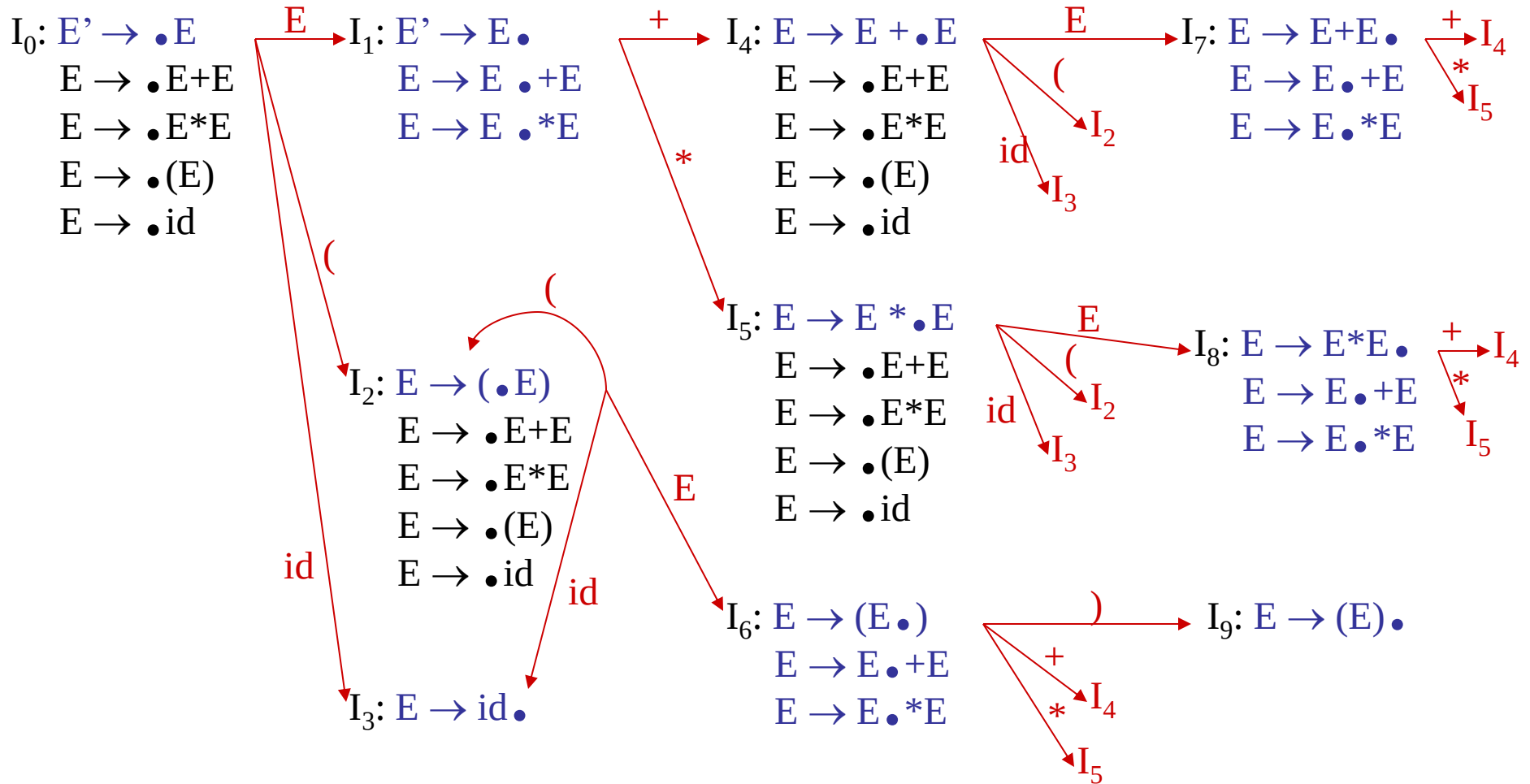
	id	*	=	\$	S	L	R
0	s5	s4			1	2	3
1				acc			
2			s6	r5			
3				r2			
4	no shift/reduce or					8	7
5	no reduce/reduce conflict						
6	s12	s11				10	9
7				r3	r3		
8	so, it is a LALR(1) grammar						
9				r1			
10				r5			
11	s12	s11				10	13
12				r4			
13				r3			

Using Ambiguous Grammars

- ◆ All grammars used in the construction of LR-parsing tables must be un-ambiguous.
- ◆ Can we create LR-parsing tables for ambiguous grammars ?
 - Yes, but they will have conflicts.
 - We can resolve these conflicts in favor of one of them to disambiguate the grammar.
 - At the end, we will have again an unambiguous grammar.
- ◆ Why we want to use an ambiguous grammar?
 - Some of the ambiguous grammars are much natural, and a corresponding unambiguous grammar can be very complex.
 - Usage of an ambiguous grammar may eliminate unnecessary reductions.
- ◆ Ex.

$$\begin{array}{lcl} E \rightarrow E+E & | & E^*E & | & (E) & | & id & \rightarrow & \begin{array}{l} E \rightarrow E+T & | & T \\ T \rightarrow T^*F & | & F \\ F \rightarrow (E) & | & id \end{array} \end{array}$$

Sets of LR(0) Items for Ambiguous Grammar



SLR-Parsing Tables for Ambiguous Grammar

$$\text{FOLLOW}(E) = \{ \$, +, *,) \}$$

State I_7 has shift/reduce conflicts for symbols $+$ and $*$.

$$I_0 \xrightarrow{E} I_1 \xrightarrow{+} I_4 \xrightarrow{E} I_7$$

when current token is $+$

shift $\rightarrow$ $+$ is right-associative

reduce $\rightarrow$ $+$ is left-associative

when current token is $*$

shift $\rightarrow$ $*$ has higher precedence than $+$

reduce $\rightarrow$ $+$ has higher precedence than $*$

SLR-Parsing Tables for Ambiguous Grammar

$$\text{FOLLOW}(E) = \{ \$, +, *,) \}$$

State I_8 has shift/reduce conflicts for symbols $+$ and $*$.

$$I_0 \xrightarrow{E} I_1 \xrightarrow{*} I_5 \xrightarrow{E} I_8$$

when current token is $*$

shift $\rightarrow$ $*$ is right-associative

reduce $\rightarrow$ $*$ is left-associative

when current token is $+$

shift $\rightarrow$ $+$ has higher precedence than $*$

reduce $\rightarrow$ $*$ has higher precedence than $+$

SLR-Parsing Tables for Ambiguous Grammar

	Action						Goto	
	id	+	*	(	)	\$		E
0	s3			s2				1
1		s4	s5			acc		
2	s3			s2				6
3		r4	r4		r4	r4		
4	s3			s2				7
5	s3			s2				8
6		s4	s5		s9			
7		r1	s5		r1	r1		
8		r2	r2		r2	r2		
9		r3	r3		r3	r3		

Error Recovery in LR Parsing

- ◆ An LR parser will detect an error when it consults the parsing action table and finds an error entry. All empty entries in the action table are error entries.
- ◆ Errors are never detected by consulting the goto table.
- ◆ An LR parser will announce error as soon as there is no valid continuation for the scanned portion of the input.
- ◆ A canonical LR parser (LR(1) parser) will never make even a single reduction before announcing an error.
- ◆ The SLR and LALR parsers may make several reductions before announcing an error.
- ◆ But, all LR parsers (LR(1), LALR and SLR parsers) will never shift an erroneous input symbol onto the stack.

Panic Mode Error Recovery in LR Parsing

- ◆ Scan down the stack until a state s with a goto on a particular nonterminal A is found. (Get rid of everything from the stack before this state s).
- ◆ Discard zero or more input symbols until a symbol a is found that can legitimately follow A .
 - The symbol a is simply in $\text{FOLLOW}(A)$, but this may not work for all situations.
- ◆ The parser stacks the nonterminal A and the state $\text{goto}[s, A]$, and it resumes the normal parsing.
- ◆ This nonterminal A is normally is a basic programming block (there can be more than one choice for A).
 - stmt, expr, block, ...

Phrase-Level Error Recovery in LR Parsing

- ◆ Each empty entry in the action table is marked with a specific error routine.
- ◆ An error routine reflects the error that the user most likely will make in that case.
- ◆ An error routine inserts the symbols into the stack or the input (or it deletes the symbols from the stack and the input, or it can do both insertion and deletion).
 - missing operand
 - unbalanced right parenthesis

小结

- ◆ 理解自底向上语法分析器的工作原理——采用栈实现的移进-规约语法分析方法，掌握句柄（**handle**）的概念。
- ◆ 理解算符优先的语法分析方法。
- ◆ 掌握LR语法分析方法——**SLR、LR、LALR**，理解**closure**操作和**goto**操作，并能够构造**SLR(1)、LR(1)、LALR(1)**语法分析表。